Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Test (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Sawan Kumar **1BF24EC136**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

INDEX

INTRODUCTION **3**
MODULE 1: 4-BIT ARITHMETIC LOGIC UNIT (ALU) **4**
Design Description 4
Circuit Diagram 5
Verilog HDL Code 5
Testbench 5
Simulation Waveform 6
MODULE 2: 4-BIT COMPARATOR **7**
Design Description 7
Circuit Diagram 7
Verilog HDL Code 7
Testbench 7
Simulation Waveform 8
MODULE 3: 4-BIT PARITY GENERATOR AND CHECKER **9**
Design Description 9
Circuit Diagram 9
Verilog HDL Code 9
Testbench 10
Simulation Waveform 10
MODULE 4: 4-BIT BARREL SHIFTER **11**
Design Description 11
Circuit Diagram 11
Verilog HDL Code 11
Testbench 12
Simulation Waveform 12
MODULE 5: TIMER MODULE **13**
Design Description 13
Circuit Diagram 13
Verilog HDL Code 13
Testbench 14
Simulation Waveform 15
MODULE 6: 4-BIT RING COUNTER **16**
Design Description 16
Circuit Diagram 16
Verilog HDL Code 16
Testbench 17
Simulation Waveform 17
MODULE 7: 4-BIT JOHNSON COUNTER **18**
Design Description 18
Circuit Diagram 18
Verilog HDL Code 18
Testbench 19

Simulation Waveform 19
MODULE 8: SYNCHRONOUS COUNTER (4-BIT UP/DOWN) 20
Design Description 20
Circuit Diagram 20
Verilog HDL Code 20
Testbench 21
Simulation Waveform 21
MODULE 9: ASYNCHRONOUS (RIPPLE) COUNTER 23
Design Description 23
Circuit Diagram 23
Verilog HDL Code 23
Testbench 24
Simulation Waveform 24
MODULE 10: 4-BIT SHIFT REGISTER (UNIVERSAL) 25
Design Description 25
Circuit Diagram 25
Verilog HDL Code 26
Testbench 26
Simulation Waveform 27
MODULE 11: PARAMETERIZED DIVIDE-BY-N COUNTER 28
Design Description 28
Circuit Diagram 28
Verilog HDL Code 28
Testbench 29
Simulation Waveform 29
MODULE 12: MEALY FSM - SEQUENCE DETECTOR (101) 30
Design Description 30
Circuit Diagram 30
Verilog HDL Code 31
Testbench 31
Simulation Waveform 32
MODULE 13: MOORE FSM - SEQUENCE DETECTOR (101) 33
Design Description 33
Circuit Diagram 33
Verilog HDL Code 34
Testbench 34
Simulation Waveform 35
MODULE 14: RANDOM SEQUENCE GENERATOR (4-BIT LFSR) 36
Design Description 36
Circuit Diagram 36
Verilog HDL Code 36
Testbench 37
Simulation Waveform 37

INTRODUCTION

This report presents the design, Verilog HDL code, testbench and simulation waveform of fourteen digital logic modules, prepared as part of the Alternative Assessment Tool (AAT) for the course Advanced Digital and Logic Design (ADLD, 23ECSPE1AD).

Tools and Methodology

Each module is written in Verilog HDL and verified with its own testbench using behavioral simulation in Xilinx Vivado 2025.2 (10 ns clock for the sequential modules). The ALU, comparator, parity circuit, timer, asynchronous counter, LFSR and both FSMs use self-checking testbenches; the remaining modules are verified from their waveforms.

Modules Covered

No.	Module	Design file	Testbench file
1	4-Bit Arithmetic Logic Unit (ALU)	alu_4bit.v	alu_4bit_tb.v
2	4-Bit Comparator	comparator_4bit.v	comparator_4bit_tb.v
3	4-Bit Parity Generator and Checker	parity.v	parity_tb.v
4	4-Bit Barrel Shifter	barrel_shifter_4bit.v	barrel_shifter_4bit_tb.v
5	Timer Module	timer_module.v	timer_module_tb.v
6	4-Bit Ring Counter	ring_counter.v	ring_counter_tb.v
7	4-Bit Johnson Counter	johnson_counter.v	johnson_counter_tb.v
8	Synchronous Counter (4-Bit Up/Down)	counter.v	counter_tb.v
9	Asynchronous (Ripple) Counter	async_counter.v	async_counter_tb.v
10	4-Bit Shift Register (Universal)	universal_shift_reg.v	universal_shift_reg_tb.v
11	Parameterized Divide-by-N Counter	div_by_N.v	div_by_N_tb.v
12	Mealy FSM - Sequence Detector (101)	mealy_fsm.v	mealy_fsm_tb.v
13	Moore FSM - Sequence Detector (101)	moore_fsm.v	moore_fsm_tb.v
14	Random Sequence Generator (4-Bit LFSR)	lfsr_4bit.v	lfsr_4bit_tb.v

MODULE 1: 4-BIT ARITHMETIC LOGIC UNIT (ALU)

Design Description

The ALU is a combinational block that performs arithmetic and logical operations on two 4-bit operands A and B, selected by the 3-bit input op. It produces a 4-bit result, a carry/borrow flag and a zero flag.

Input / output ports

Port	Direction	Width	Description
a[3:0]	Input	4	Operand A
b[3:0]	Input	4	Operand B
op[2:0]	Input	3	Operation select
result[3:0]	Output	4	Result of the selected operation
carry	Output	1	Carry-out / borrow / bit shifted out
zero	Output	1	High when result = 0000

Operation select table

op	Operation	Description
000	ADD	result = A + B
001	SUB	result = A - B (carry = borrow)
010	AND	result = A & B
011	OR	result = A | B
100	XOR	result = A ^ B
101	NOT	result = ~A
110	SHL	A shifted left by 1
111	SHR	A shifted right by 1

Circuit Diagram

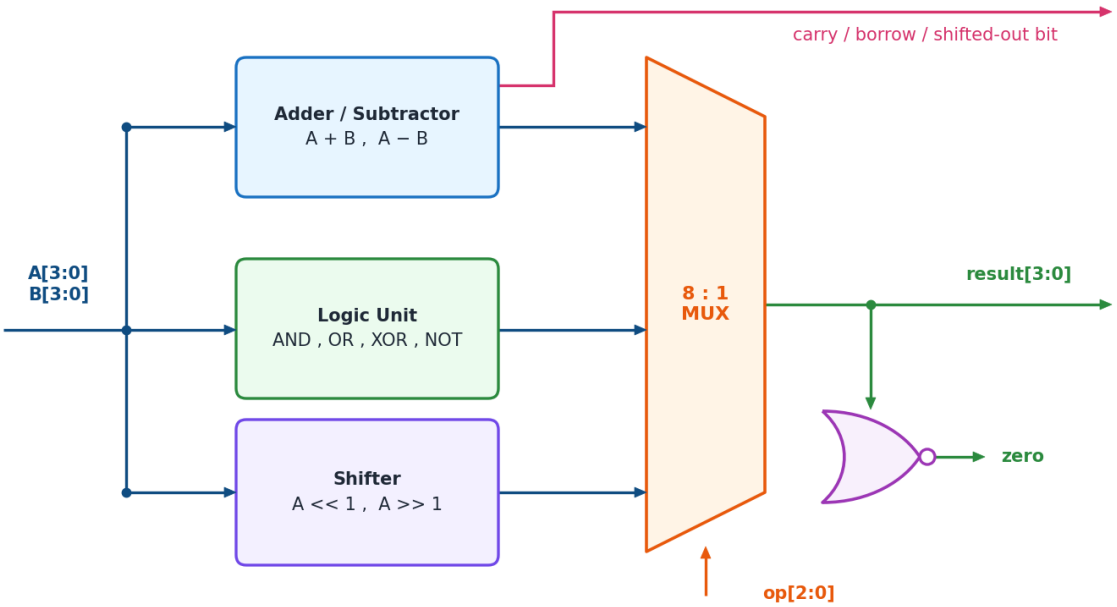


Figure 1.1: Block diagram of the 4-Bit Arithmetic Logic Unit (ALU)

Verilog HDL Code

```
`timescale 1ns / 1ps

module alu_4bit(
    input  [3:0] a, b,
    input  [2:0] op,
    output reg [3:0] result,
    output reg carry,       // carry-out (ADD), borrow (SUB), bit shifted out (shifts)
    output zero
);
    assign zero = (result == 4'b0000);

    always @(*) begin
        carry  = 1'b0;
        result = 4'b0000;
        case (op)
            3'b000: {carry, result} = a + b;                  // ADD
            3'b001: {carry, result} = {1'b0, a} - {1'b0, b};  // SUB (carry = borrow)
            3'b010: result = a & b;                           // AND
            3'b011: result = a | b;                           // OR
            3'b100: result = a ^ b;                           // XOR
            3'b101: result = ~a;                              // NOT A
            3'b110: {carry, result} = {a, 1'b0};              // shift A left by 1
            3'b111: {result, carry} = {1'b0, a};              // shift A right by 1
            default: result = 4'b0000;
        endcase
    end
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module alu_4bit_tb;
    reg  [3:0] a, b;
    reg  [2:0] op;
    wire [3:0] result;
    wire carry, zero;
    integer errors;

    alu_4bit DUT(.a(a), .b(b), .op(op), .result(result), .carry(carry), .zero(zero));
```

```
    task check(input [3:0] exp_result, input exp_carry);
    begin
        #10;
        if (result !== exp_result || carry !== exp_carry) begin
            $display("FAIL: op=%b a=%b b=%b -> result=%b carry=%b (expected %b %b)",
                     op, a, b, result, carry, exp_result, exp_carry);
            errors = errors + 1;
        end else
            $display("PASS: op=%b a=%b b=%b -> result=%b carry=%b", op, a, b, result, carry);
    end
    endtask

    initial begin
        errors = 0;
        $dumpfile("alu_4bit.vcd");
        $dumpvars(0, alu_4bit_tb);
        $monitor("t=%0t op=%b a=%b b=%b result=%b carry=%b zero=%b",
                 $time, op, a, b, result, carry, zero);

        a = 4'b0110; b = 4'b0011;                  // a = 6, b = 3
        op = 3'b000; check(4'b1001, 1'b0);         // ADD  6+3 = 9
        op = 3'b001; check(4'b0011, 1'b0);         // SUB  6-3 = 3
        op = 3'b010; check(4'b0010, 1'b0);         // AND
        op = 3'b011; check(4'b0111, 1'b0);         // OR
        op = 3'b100; check(4'b0101, 1'b0);         // XOR
        op = 3'b101; check(4'b1001, 1'b0);         // NOT A
        op = 3'b110; check(4'b1100, 1'b0);         // shift left
        op = 3'b111; check(4'b0011, 1'b0);         // shift right

        a = 4'b1111; b = 4'b0001; op = 3'b000;     // ADD overflow: 15+1
        check(4'b0000, 1'b1);
        if (zero !== 1'b1) begin $display("FAIL: zero flag"); errors = errors + 1; end

        a = 4'b0011; b = 4'b0110; op = 3'b001;     // SUB with borrow: 3-6
        check(4'b1101, 1'b1);

        if (errors == 0) $display("ALL TESTS PASSED");
        else $display("TESTS FAILED: %0d", errors);
        $finish;
    end
endmodule
```

Simulation Waveform

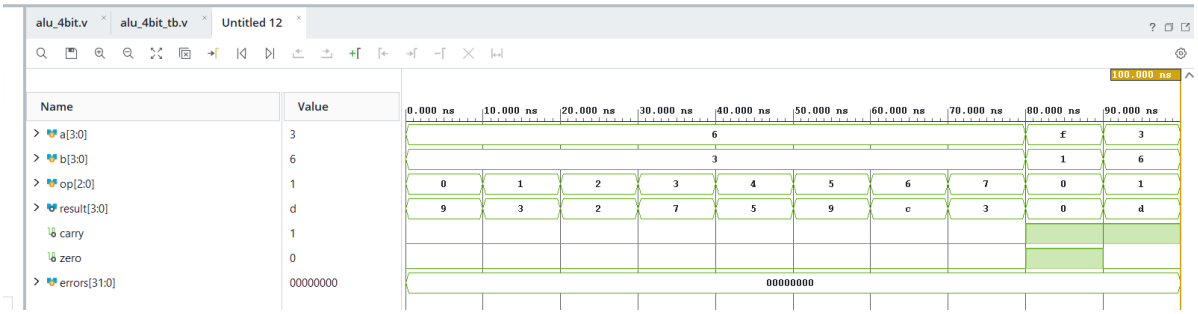


Figure 1.2: Simulation waveform of the 4-Bit Arithmetic Logic Unit (ALU)

Observation: With A = 0110 (6) and B = 0011 (3), the waveform shows the results 9, 3, 2, 7, 5, 9, C and 3 for op = 000 to 111 respectively, which match the expected ADD, SUB, AND, OR, XOR, NOT, shift-left and shift-right outputs. For A = 1111 and B = 0001 the addition gives 0000 with carry = 1 and zero = 1, and for A = 0011 and B = 0110 the subtraction gives 1101 with the borrow flag set. The self-checking testbench reports errors = 0.

MODULE 2: 4-BIT COMPARATOR

Design Description

The comparator compares two 4-bit numbers A and B and drives exactly one of three outputs: gt (A > B), eq (A = B) or lt (A < B). It is purely combinational.

Input / output ports

Port	Direction	Width	Description
a[3:0]	Input	4	Operand A
b[3:0]	Input	4	Operand B
gt	Output	1	High when A > B
eq	Output	1	High when A = B
lt	Output	1	High when A < B

Circuit Diagram

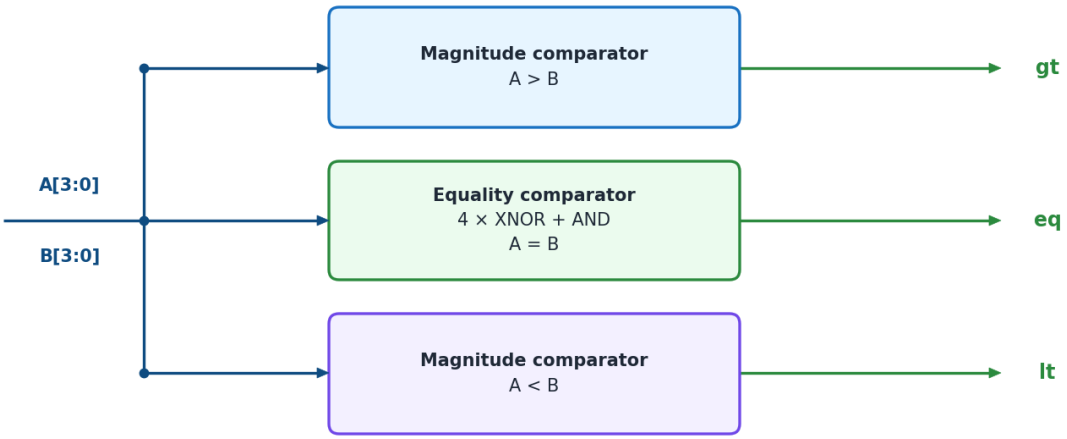


Figure 2.1: Block diagram of the 4-Bit Comparator

Verilog HDL Code

```
`timescale 1ns / 1ps

module comparator_4bit(
    input  [3:0] a, b,
    output gt,   // a > b
    output eq,   // a == b
    output lt    // a < b
);
    assign gt = (a > b);
    assign eq = (a == b);
    assign lt = (a < b);
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module comparator_4bit_tb;
    reg  [3:0] a, b;
    wire gt, eq, lt;
```

```
    integer i, j, errors;

    comparator 4bit DUT(.a(a), .b(b), .gt(gt), .eq(eq), .lt(lt));

    initial begin
        errors = 0;
        $dumpfile("comparator_4bit.vcd");
        $dumpvars(0, comparator 4bit tb);
        $monitor("t=%0t a=%d b=%d gt=%b eq=%b lt=%b", $time, a, b, gt, eq, lt);

        // Exhaustive test: all 16 x 16 input combinations
        for (i = 0; i < 16; i = i + 1) begin
            for (j = 0; j < 16; j = j + 1) begin
                a = i; b = j; #5;
                if (gt !== (i > j) || eq !== (i == j) || lt !== (i < j)) begin
                    $display("FAIL: a=%0d b=%0d gt=%b eq=%b lt=%b", i, j, gt, eq, lt);
                    errors = errors + 1;
                end
            end
        end

        if (errors == 0) $display("ALL TESTS PASSED");
        else $display("TESTS FAILED: %0d", errors);
        $finish;
    end
endmodule
```

Simulation Waveform

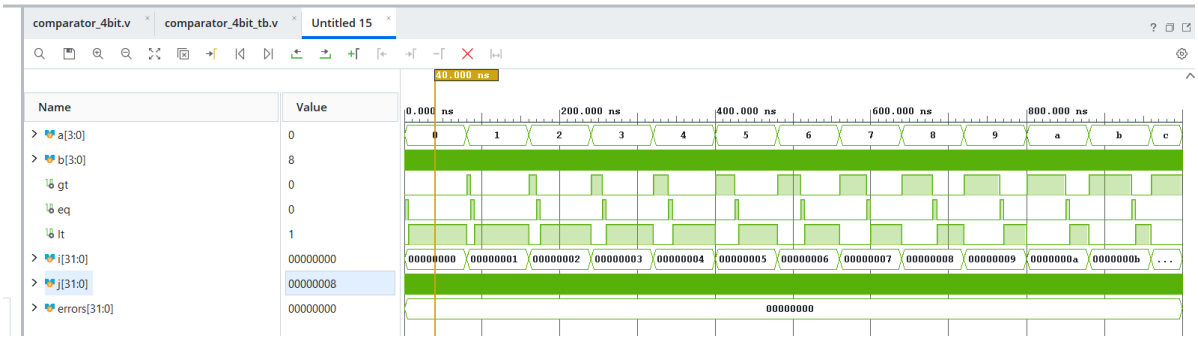


Figure 2.2: Simulation waveform of the 4-Bit Comparator

Observation: In the waveform, A is held while B sweeps through all sixteen values, then A increments and the sweep repeats. At every instant exactly one of gt, eq and lt is high, and eq pulses only when A equals B. The testbench checks all 256 combinations and the error counter remains 0.

MODULE 3: 4-BIT PARITY GENERATOR AND CHECKER

Design Description

Even parity is used. The generator XORs the four data bits to produce the parity bit; the checker XORs the received data and parity bit, so error = 1 indicates a single-bit error.

Input / output ports

Port	Direction	Width	Description
data[3:0] (generator)	Input	4	Data word
parity (generator)	Output	1	Even-parity bit
data[3:0] (checker)	Input	4	Received data word
parity_in (checker)	Input	1	Received parity bit
error (checker)	Output	1	1 = parity error detected

Circuit Diagram

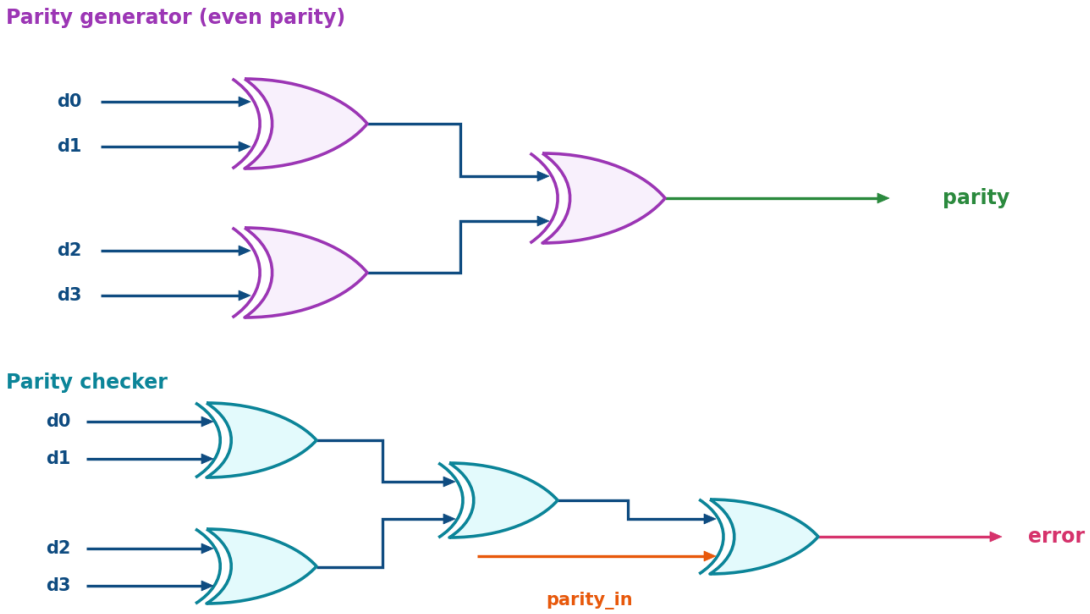


Figure 3.1: Circuit diagram of the 4-Bit Parity Generator and Checker

Verilog HDL Code

```
`timescale 1ns / 1ps

// Even-parity generator: parity bit makes the total number of 1s even
module parity generator 4bit(
    input  [3:0] data,
    output parity
);
    assign parity = ^data;
endmodule

// Even-parity checker: error = 1 when data + parity bit contain an odd number of 1s
module parity checker 4bit(
    input  [3:0] data,
    input  parity_in,
```

```
    output error
);
    assign error = ^{parity in, data};
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module parity tb;
    reg  [3:0] data;
    reg  flip;                  // 1 = inject a single-bit error on the parity line
    wire parity tx, parity rx, error;
    integer i, errors;

    parity_generator_4bit GEN(.data(data), .parity(parity_tx));
    assign parity_rx = parity_tx ^ flip;
    parity checker 4bit  CHK(.data(data), .parity in(parity rx), .error(error));

    initial begin
        errors = 0; flip = 0;
        $dumpfile("parity.vcd");
        $dumpvars(0, parity_tb);
        $monitor("t=%0t data=%b parity_tx=%b flip=%b parity_rx=%b error=%b",
                 $time, data, parity tx, flip, parity rx, error);

        for (i = 0; i < 16; i = i + 1) begin
            data = i; flip = 0; #5;               // error-free transmission
            if (parity_tx !== (^data) || error !== 1'b0) begin
                $display("FAIL (no error case): data=%b", data); errors = errors + 1;
            end
            flip = 1; #5;                         // corrupted parity bit
            if (error !== 1'b1) begin
                $display("FAIL (error not detected): data=%b", data); errors = errors + 1;
            end
        end

        if (errors == 0) $display("ALL TESTS PASSED");
        else $display("TESTS FAILED: %0d", errors);
        $finish;
    end
endmodule
```

Simulation Waveform



Figure 3.2: Simulation waveform of the 4-Bit Parity Generator and Checker

Observation: All sixteen data words are applied. When flip = 0, parity_tx is 1 only for the words having an odd number of 1s (1, 2, 4, 7, 8, B, D and E) and error stays 0. When flip = 1 the received parity bit is inverted and the checker output error goes high for every data word, so every injected single-bit error is detected. The testbench reports errors = 0.

MODULE 4: 4-BIT BARREL SHIFTER

Design Description

The barrel shifter shifts or rotates a 4-bit word by 0 to 3 positions in one combinational step. The mode input selects the operation and shamt gives the shift amount.

Input / output ports

Port	Direction	Width	Description
data[3:0]	Input	4	Input data
shamt[1:0]	Input	2	Shift amount (0 to 3)
mode[1:0]	Input	2	Operation select
out[3:0]	Output	4	Shifted / rotated output

Mode select table

mode	Operation
00	Logical shift left
01	Logical shift right
10	Rotate left
11	Rotate right

Circuit Diagram

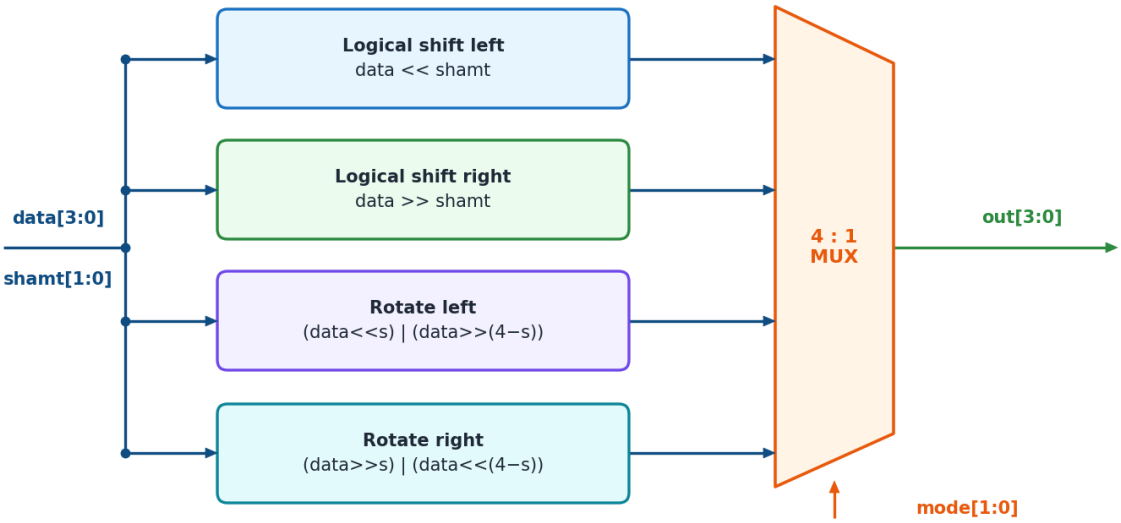


Figure 4.1: Block diagram of the 4-Bit Barrel Shifter

Verilog HDL Code

```
`timescale 1ns / 1ps

module barrel_shifter_4bit(
    input [3:0] data,
    input [1:0] shamt,
```

```
    input [1:0] mode,
    output reg [3:0] out
);
    always @(*) begin
        case (mode)
            2'b00: out = data << shamt; // logical left
            2'b01: out = data >> shamt; // logical right
            2'b10: out = (data << shamt) | (data >> (4-shamt)); // rotate left
            2'b11: out = (data >> shamt) | (data << (4-shamt)); // rotate right
            default: out = data;
        endcase
    end
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module barrel_shifter_4bit_tb;
    reg [3:0] data;
    reg [1:0] shamt, mode;
    wire [3:0] out;

    barrel_shifter_4bit DUT(.data(data), .shamt(shamt), .mode(mode), .out(out));

    initial begin
        $dumpfile("barrel shifter.vcd");
        $dumpvars(0, barrel_shifter_4bit_tb);
        $monitor("t=%0t data=%b shamt=%d mode=%b out=%b",
                 $time, data, shamt, mode, out);

        data = 4'b1011; shamt = 2; mode = 0; #10;
        mode = 1; #10;
        mode = 2; #10;
        mode = 3; #10;
        shamt = 0; mode = 2; #10;
        $finish;
    end
endmodule
```

Simulation Waveform

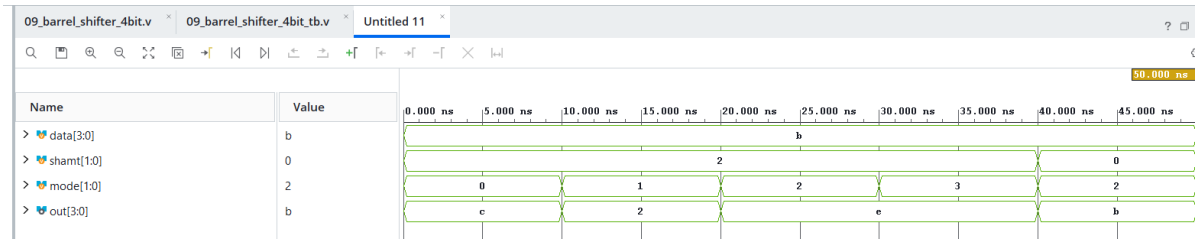


Figure 4.2: Simulation waveform of the 4-Bit Barrel Shifter

Observation: For data = 1011 and shamt = 2 the output is 1100 (C) for logical left, 0010 (2) for logical right and 1110 (E) for both rotate left and rotate right (rotating a 4-bit word by 2 positions gives the same result in either direction). When shamt = 0 with rotate left the data passes through unchanged (B).

MODULE 5: TIMER MODULE

Design Description

The timer is a programmable down counter. When start is applied while idle, load_val is loaded and counted down once per clock; after load_val clock cycles, done goes high for one clock cycle.

Input / output ports

Port	Direction	Width	Description
clk	Input	1	Clock
reset	Input	1	Synchronous active-high reset
start	Input	1	Starts the timer when idle
load_val[3:0]	Input	4	Number of clock cycles to count
remaining[3:0]	Output	4	Remaining count
running	Output	1	High while the timer is counting
done	Output	1	One-cycle pulse when the time has elapsed

Circuit Diagram

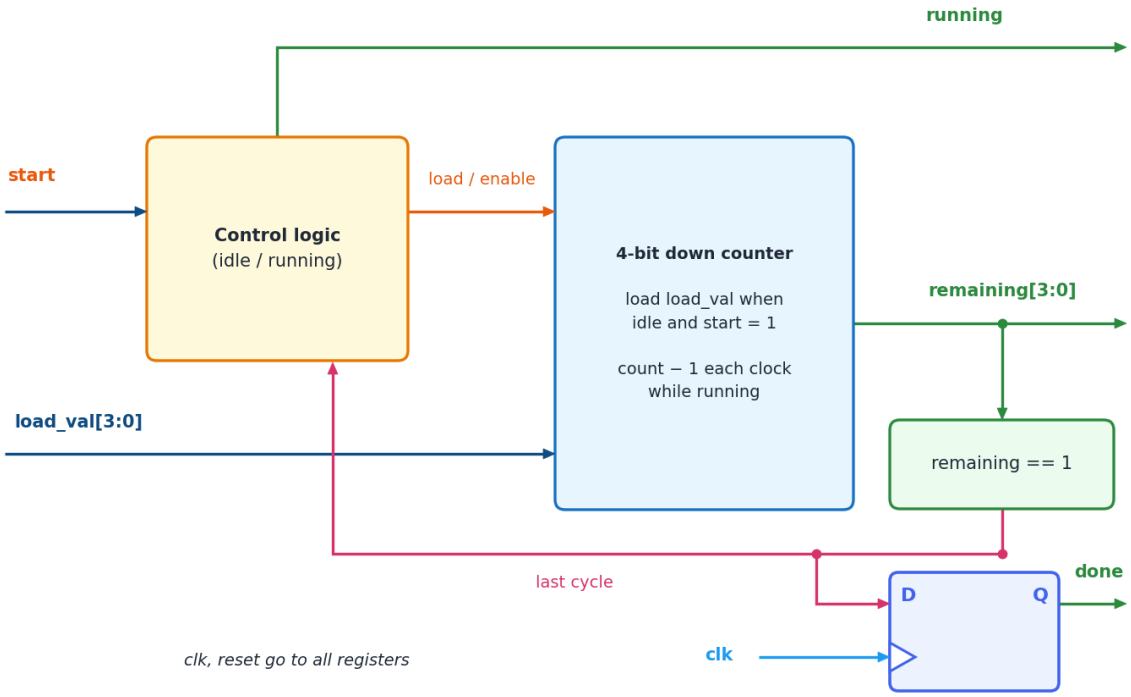


Figure 5.1: Block diagram of the Timer Module

Verilog HDL Code

```
`timescale 1ns / 1ps

// Programmable down-counting timer.
// start=1 (while idle) loads load_val; the timer then counts down once per clock.
// done is a one-clock pulse when the count has expired.
```

```
module timer module(
    input  clk, reset, start,
    input  [3:0] load val,
    output reg [3:0] remaining,
    output reg running,
    output reg done
);
    always @(posedge clk) begin
        if (reset) begin
            remaining <= 4'd0;
            running   <= 1'b0;
            done      <= 1'b0;
        end else begin
            done <= 1'b0;                          // default: pulse lasts one cycle
            if (!running) begin
                if (start && (load_val != 4'd0)) begin
                    remaining <= load_val;
                    running   <= 1'b1;
                end
            end else begin
                if (remaining == 4'd1) begin
                    remaining <= 4'd0;
                    running   <= 1'b0;
                    done      <= 1'b1;
                end else
                    remaining <= remaining - 1'b1;
            end
        end
    end
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module timer module tb;
    reg clk, reset, start;
    reg [3:0] load_val;
    wire [3:0] remaining;
    wire running, done;
    integer errors;

    timer_module DUT(.clk(clk), .reset(reset), .start(start), .load_val(load_val),
                     .remaining(remaining), .running(running), .done(done));
    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1; start = 0; load_val = 0; errors = 0;
        $dumpfile("timer module.vcd");
        $dumpvars(0, timer_module_tb);
        $monitor("t=%0t reset=%b start=%b load=%0d remaining=%0d running=%b done=%b",
                 $time, reset, start, load_val, remaining, running, done);

        repeat (2) @(negedge clk);
        reset = 0;

        // Test 1: 3-cycle timer
        load_val = 4'd3; start = 1; @(negedge clk);
        start = 0;
        repeat (2) @(negedge clk); #1;
        if (done !== 1'b0 || remaining !== 4'd1) begin
            $display("FAIL: timer expired early"); errors = errors + 1;
        end
        @(negedge clk); #1;
        if (done !== 1'b1 || remaining !== 4'd0) begin
            $display("FAIL: done not asserted after 3 cycles"); errors = errors + 1;
        end else $display("PASS: 3-cycle timer");
        @(negedge clk); #1;
        if (done !== 1'b0) begin
            $display("FAIL: done pulse longer than one cycle"); errors = errors + 1;
        end

        // Test 2: 1-cycle timer
        load_val = 4'd1; start = 1; @(negedge clk);
        start = 0; @(negedge clk); #1;
```

```
        if (done !== 1'b1) begin
            $display("FAIL: 1-cycle timer"); errors = errors + 1;
        end else $display("PASS: 1-cycle timer");

        if (errors == 0) $display("ALL TESTS PASSED");
        else $display("TESTS FAILED: %0d", errors);
        $finish;
    end
endmodule
```

Simulation Waveform

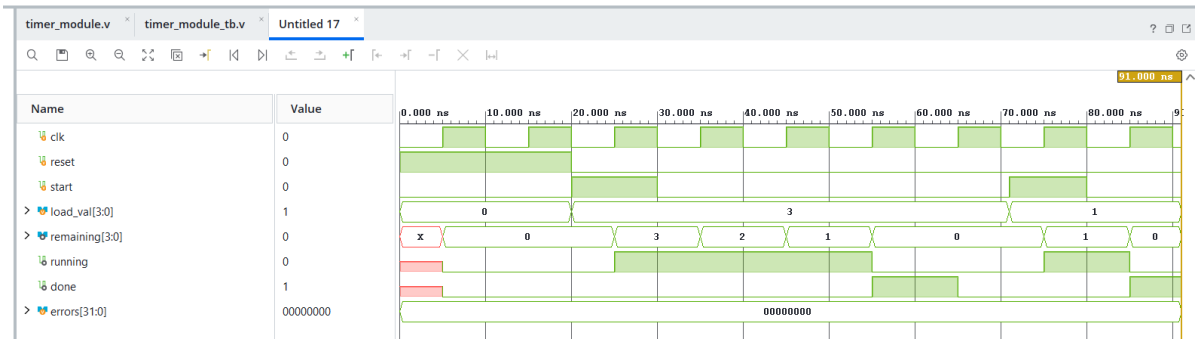


Figure 5.2: Simulation waveform of the Timer Module

Observation: For load_val = 3 the remaining count goes 3, 2, 1, 0 on successive clock edges, running stays high during the count and done pulses for one clock cycle when the count reaches 0. A second run with load_val = 1 shows remaining = 1 followed immediately by done. The testbench reports errors = 0.

MODULE 6: 4-BIT RING COUNTER

Design Description

A ring counter circulates a single 1 through four D flip-flops by feeding Q3 back to D0. Reset initialises the register to 0001, giving a one-hot, four-state sequence.

Input / output ports

Port	Direction	Width	Description
clk	Input	1	Clock
reset	Input	1	Synchronous reset (loads 0001)
count[3:0]	Output	4	One-hot count

Circuit Diagram

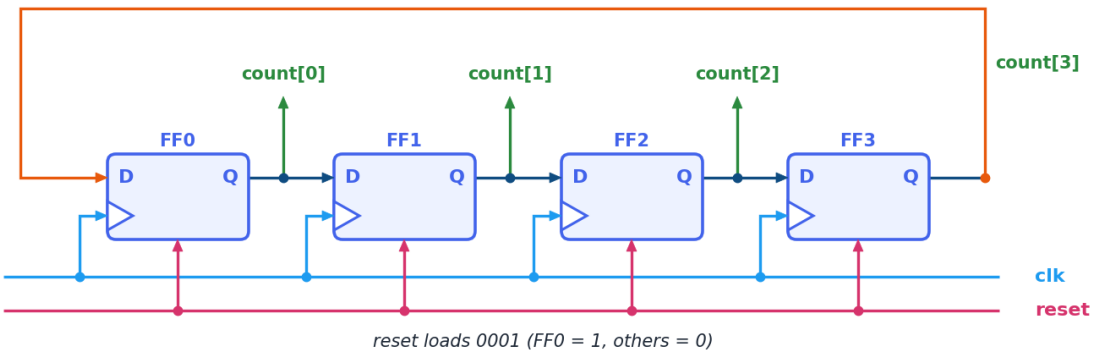


Figure 6.1: Circuit diagram of the 4-Bit Ring Counter

Verilog HDL Code

```
`timescale 1ns / 1ps

module ring_counter(
    input clk, reset,
    output [3:0] count
);
    d_ff_ring d1(.clk(clk), .reset(reset), .d(count[3]), .q(count[0]), .reset_value(1'b1));
    d_ff_ring d2(.clk(clk), .reset(reset), .d(count[0]), .q(count[1]), .reset_value(1'b0));
    d_ff_ring d3(.clk(clk), .reset(reset), .d(count[1]), .q(count[2]), .reset_value(1'b0));
    d_ff_ring d4(.clk(clk), .reset(reset), .d(count[2]), .q(count[3]), .reset_value(1'b0));
endmodule

module d_ff_ring(
    input clk, reset, d, reset_value,
    output reg q
);
    always @(posedge clk) begin
        if (reset) q <= reset_value;
        else       q <= d;
    end
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module ring_counter_tb;
    reg clk, reset;
    wire [3:0] count;

    ring_counter DUT(.clk(clk), .reset(reset), .count(count));
    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1;
        $dumpfile("ring_counter.vcd");
        $dumpvars(0, ring_counter_tb);
        $monitor("t=%0t reset=%b count=%b", $time, reset, count);

        repeat (2) @(negedge clk);
        reset = 0;
        repeat (10) @(negedge clk);
        $finish;
    end
endmodule
```

Simulation Waveform

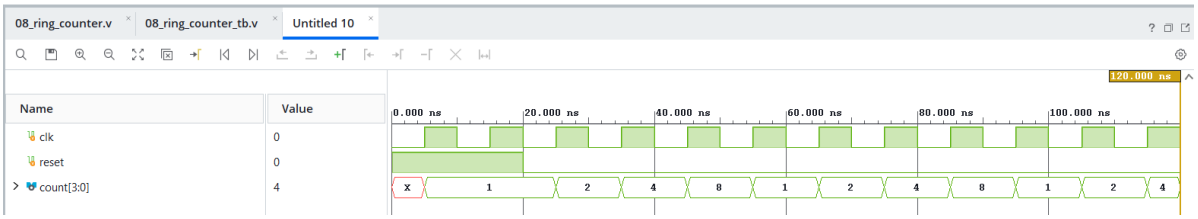


Figure 6.2: Simulation waveform of the 4-Bit Ring Counter

Observation: After reset is released the output shows the repeating sequence 0001, 0010, 0100, 1000 (1, 2, 4, 8 in hexadecimal), with the 1 moving one position to the left on every rising clock edge and returning to 0001 after four clocks.

MODULE 7: 4-BIT JOHNSON COUNTER

Design Description

A Johnson counter feeds the complement of the last flip-flop back to the first, giving 8 states for four flip-flops with only one bit changing per clock. Reset loads 0000.

Input / output ports

Port	Direction	Width	Description
clk	Input	1	Clock
reset	Input	1	Synchronous reset (loads 0000)
count[3:0]	Output	4	Johnson count

Circuit Diagram

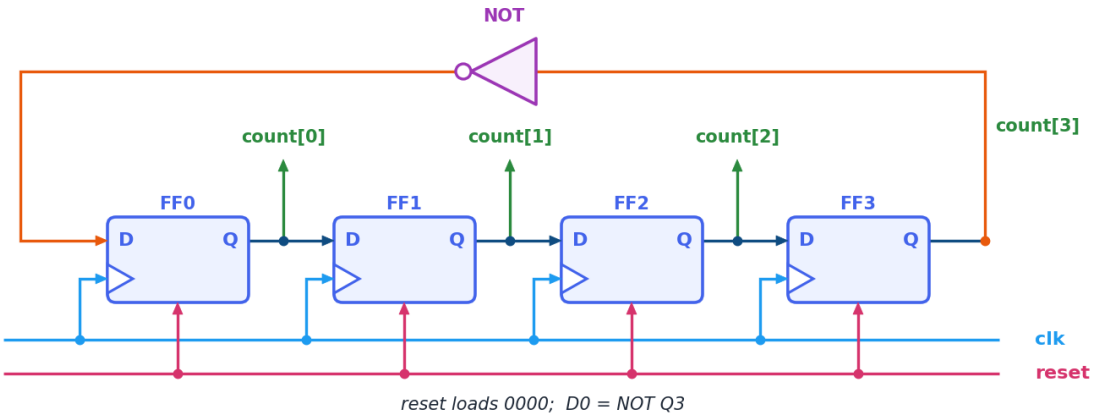


Figure 7.1: Circuit diagram of the 4-Bit Johnson Counter

Verilog HDL Code

```
`timescale 1ns / 1ps

module johnson_counter(
    input clk, reset,
    output [3:0] count
);
    d_ff d1(.clk(clk), .reset(reset), .d(~count[3]), .q(count[0]));
    d_ff d2(.clk(clk), .reset(reset), .d(count[0]), .q(count[1]));
    d_ff d3(.clk(clk), .reset(reset), .d(count[1]), .q(count[2]));
    d_ff d4(.clk(clk), .reset(reset), .d(count[2]), .q(count[3]));
endmodule

module d_ff(
    input clk, reset, d,
    output reg q
);
    always @(posedge clk) begin
        if (reset) q <= 1'b0;
        else       q <= d;
    end
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module johnson counter tb;
    reg clk, reset;
    wire [3:0] count;

    johnson counter DUT(.clk(clk), .reset(reset), .count(count));
    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1;
        $dumpfile("johnson counter.vcd");
        $dumpvars(0, johnson counter tb);
        $monitor("t=%0t reset=%b count=%b", $time, reset, count);

        repeat (2) @(negedge clk);
        reset = 0;
        repeat (10) @(negedge clk);
        $finish;
    end
endmodule
```

Simulation Waveform



Figure 7.2: Simulation waveform of the 4-Bit Johnson Counter

Observation: The output follows the sequence 0000, 0001, 0011, 0111, 1111, 1110, 1100, 1000 (0, 1, 3, 7, F, E, C, 8 in hexadecimal) and then repeats from 0000, confirming the eight-state cycle.

MODULE 8: SYNCHRONOUS COUNTER (4-BIT UP/DOWN)

Design Description

In a synchronous counter all flip-flops share the same clock. This 4-bit up/down counter has one state register; up_counter and down_counter compute the next value and mode (1 = up, 0 = down) selects between them.

Input / output ports

Port	Direction	Width	Description
clk	Input	1	Clock
reset	Input	1	Synchronous active-high reset
mode	Input	1	1 = count up, 0 = count down
count[3:0]	Output	4	Counter value

Circuit Diagram

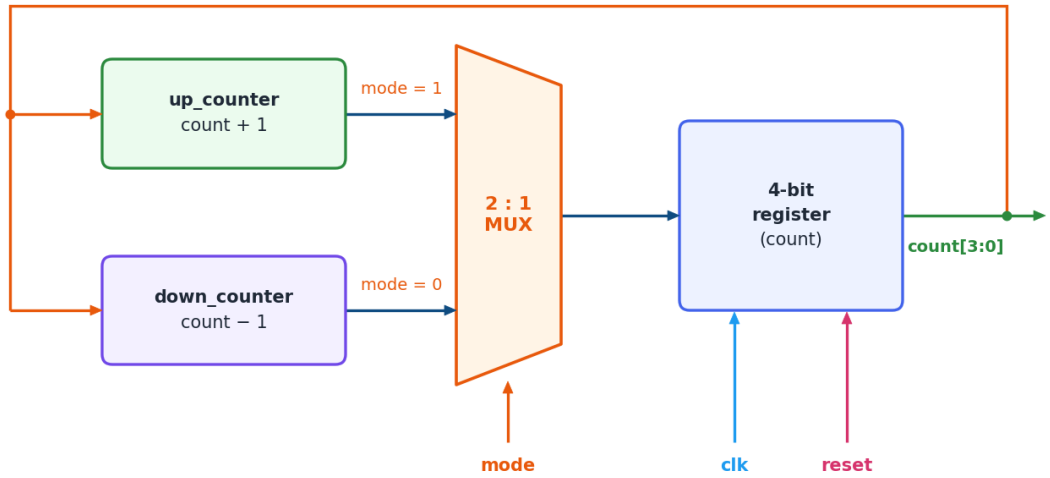


Figure 8.1: Block diagram of the Synchronous Counter (4-Bit Up/Down)

Verilog HDL Code

```
`timescale 1ns / 1ps

module counter(
    input clk,
    input reset,
    input mode,              // 1 = up, 0 = down
    output reg [3:0] count
);
    wire [3:0] up_count;
    wire [3:0] down_count;

    up_counter U1(.count(count), .next_count(up_count));
    down_counter D1(.count(count), .next_count(down_count));

    always @(posedge clk) begin
        if (reset)
            count <= 4'b0000;
        else if (mode)
```

```
            count <= up_count;
        else
            count <= down_count;
    end
endmodule

module up_counter(
    input [3:0] count,
    output [3:0] next_count
);
    assign next_count = count + 1'b1;
endmodule

module down_counter(
    input [3:0] count,
    output [3:0] next_count
);
    assign next count = count - 1'b1;
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module counter_tb;
    reg clk, reset, mode;
    wire [3:0] count;

    counter DUT(.clk(clk), .reset(reset), .mode(mode), .count(count));
    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1; mode = 1;
        $dumpfile("counter.vcd");
        $dumpvars(0, counter_tb);
        $monitor("t=%0t reset=%b mode=%b count=%b", $time, reset, mode, count);

        repeat (2) @(negedge clk);
        reset = 0;

        mode = 1; // count up
        repeat (5) @(negedge clk);

        mode = 0; // count down
        repeat (7) @(negedge clk);

        reset = 1;
        @(negedge clk);
        reset = 0;
        mode = 1;
        repeat (4) @(negedge clk);

        $finish;
    end
endmodule
```

Simulation Waveform

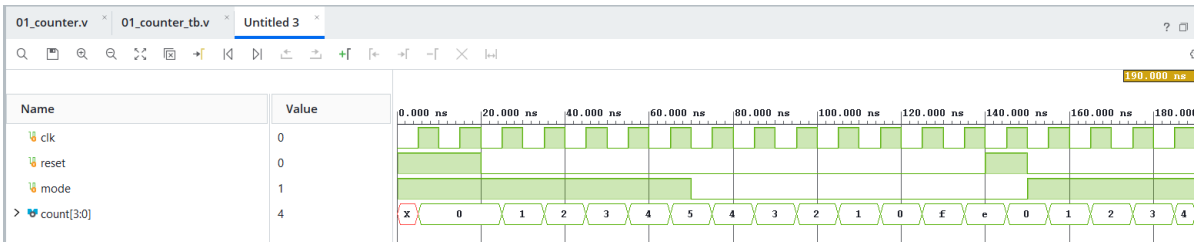


Figure 8.2: Simulation waveform of the Synchronous Counter (4-Bit Up/Down)

Observation: After reset the counter counts up from 0 to 5 while mode = 1. When mode changes to 0 it counts down 4, 3, 2, 1, 0 and wraps around to F and E. A second reset forces the count to 0, after which it counts up again (1, 2, 3, 4).

MODULE 9: ASYNCHRONOUS (RIPPLE) COUNTER

Design Description

In an asynchronous (ripple) counter only the first flip-flop receives the clock; each later stage is clocked by the previous stage output. It uses four negative-edge T flip-flops (T = 1) with an asynchronous reset.

Input / output ports

Port	Direction	Width	Description
clk	Input	1	Clock for the first stage
reset	Input	1	Asynchronous active-high reset
q[3:0]	Output	4	Counter value

Circuit Diagram

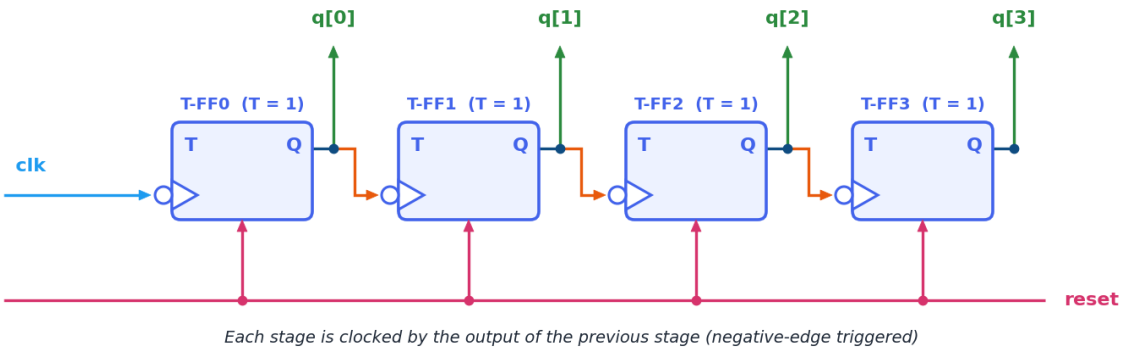


Figure 9.1: Circuit diagram of the Asynchronous (Ripple) Counter

Verilog HDL Code

```
`timescale 1ns / 1ps

// 4-bit asynchronous (ripple) up counter built from T flip-flops.
// Each stage is clocked by the output of the previous stage.
module async_counter(
    input clk, reset,
    output [3:0] q
);
    t_ff t0(.clk(clk),  .reset(reset), .q(q[0]));
    t_ff t1(.clk(q[0]), .reset(reset), .q(q[1]));
    t_ff t2(.clk(q[1]), .reset(reset), .q(q[2]));
    t_ff t3(.clk(q[2]), .reset(reset), .q(q[3]));
endmodule

// T flip-flop with T = 1 (toggles on every falling clock edge), asynchronous reset
module t_ff(
    input clk, reset,
    output reg q
);
    always @(negedge clk or posedge reset) begin
        if (reset) q <= 1'b0;
        else       q <= ~q;
    end
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module async_counter_tb;
    reg clk, reset;
    wire [3:0] q;
    integer i, errors;

    async_counter DUT(.clk(clk), .reset(reset), .q(q));
    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1; errors = 0;
        $dumpfile("async_counter.vcd");
        $dumpvars(0, async_counter_tb);
        $monitor("t=%0t reset=%b q=%b (%0d)", $time, reset, q, q);

        #12 reset = 0;                      // release between clock edges

        // Count must increase by 1 on every falling clock edge (wraps after 15)
        for (i = 1; i <= 20; i = i + 1) begin
            @(negedge clk); #1;             // allow the ripple to settle
            if (q !== i[3:0]) begin
                $display("FAIL: expected %0d got %0d", i[3:0], q); errors = errors + 1;
            end
        end

        // Asynchronous reset: clears immediately, without waiting for a clock edge
        #2 reset = 1; #1;
        if (q !== 4'b0000) begin
            $display("FAIL: async reset"); errors = errors + 1;
        end else $display("PASS: asynchronous reset");

        if (errors == 0) $display("ALL TESTS PASSED");
        else $display("TESTS FAILED: %0d", errors);
        $finish;
    end
endmodule
```

Simulation Waveform

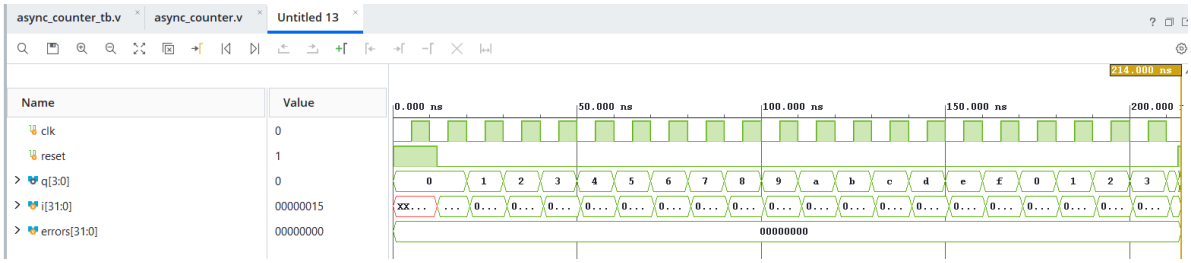


Figure 9.2: Simulation waveform of the Asynchronous (Ripple) Counter

Observation: The output increments by one on every falling edge of the clock: 0, 1, 2, ..., E, F and then wraps to 0. At the end of the run the asynchronous reset clears the counter immediately, without waiting for a clock edge. The testbench reports errors = 0.

MODULE 10: 4-BIT SHIFT REGISTER (UNIVERSAL)

Design Description

The universal shift register holds, shifts right, shifts left or loads in parallel depending on mode. Separate next-state blocks feed a 4-to-1 multiplexer in front of the state register.

Input / output ports

Port	Direction	Width	Description
clk	Input	1	Clock
reset	Input	1	Synchronous active-high reset
mode[1:0]	Input	2	Operation select
serial_in	Input	1	Serial input for shift operations
parallel_in[3:0]	Input	4	Parallel load data
Q[3:0]	Output	4	Register contents
serial_out	Output	1	Least significant bit of Q

Mode select table

mode	Operation
00	Hold
01	Shift right
10	Shift left
11	Parallel load

Circuit Diagram

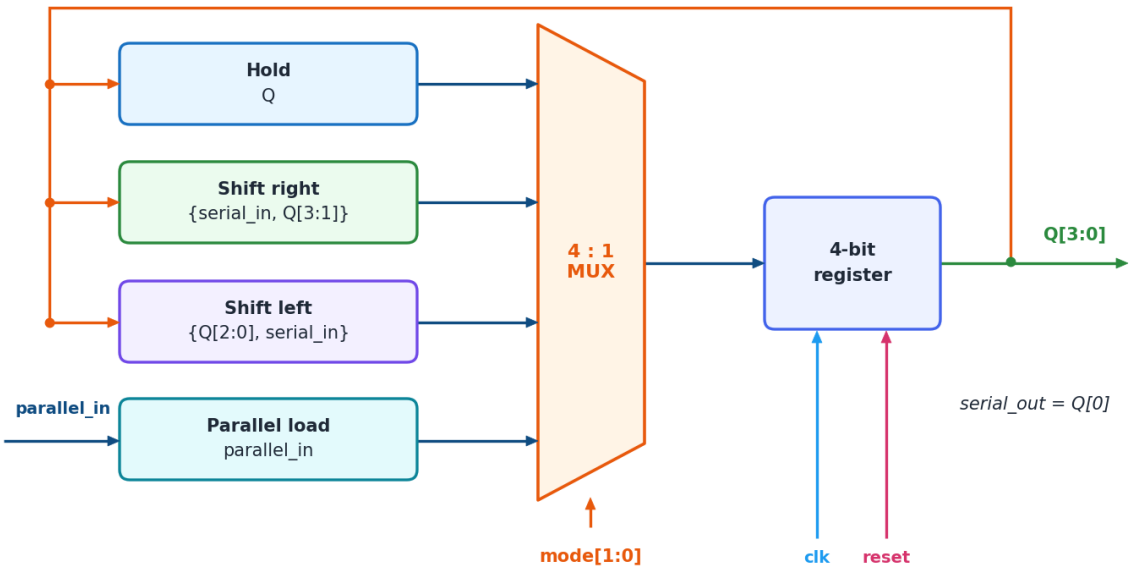


Figure 10.1: Block diagram of the 4-Bit Shift Register (Universal)

Verilog HDL Code

```
`timescale 1ns / 1ps

module universal_shift_reg_4bit(
    input clk, reset,
    input [1:0] mode,
    input serial_in,
    input [3:0] parallel_in,
    output reg [3:0] Q,
    output serial_out
);
    wire [3:0] right_data, left_data, load_data, next_Q;
    assign serial_out = Q[0];

    shift_right_4bit SR1(.Q(Q), .serial_in(serial_in), .next_Q(right_data));
    shift_left_4bit  SL1(.Q(Q), .serial_in(serial_in), .next_Q(left_data));
    load_4bit        PL1(.parallel_in(parallel_in), .next_Q(load_data));
    mux_4to1_4bit    M1(.Q(Q), .right_data(right_data),
                        .left_data(left_data), .load_data(load_data),
                        .mode(mode), .next_Q(next_Q));

    always @(posedge clk) begin
        if (reset) Q <= 4'b0000;
        else Q <= next_Q;
    end
endmodule

module shift_right_4bit(input [3:0] Q, input serial_in, output [3:0] next_Q);
    assign next_Q = {serial_in, Q[3:1]};
endmodule

module shift_left_4bit(input [3:0] Q, input serial_in, output [3:0] next_Q);
    assign next_Q = {Q[2:0], serial_in};
endmodule

module load_4bit(input [3:0] parallel_in, output [3:0] next_Q);
    assign next_Q = parallel_in;
endmodule

module mux_4to1_4bit(
    input [3:0] Q, right_data, left_data, load_data,
    input [1:0] mode,
    output reg [3:0] next_Q
);
    always @(*) begin
        case (mode)
            2'b00: next_Q = Q;
            2'b01: next_Q = right_data;
            2'b10: next_Q = left_data;
            2'b11: next_Q = load_data;
            default: next_Q = Q;
        endcase
    end
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module universal_shift_reg_tb;
    reg clk, reset;
    reg [1:0] mode;
    reg serial_in;
    reg [3:0] parallel_in;
    wire [3:0] Q;
    wire serial_out;

    universal_shift_reg_4bit DUT(
        .clk(clk), .reset(reset), .mode(mode),
        .serial_in(serial_in), .parallel_in(parallel_in),
        .Q(Q), .serial_out(serial_out)
    );
    always #5 clk = ~clk;
```

```
    initial begin
        clk = 0; reset = 1; mode = 0; serial_in = 0; parallel_in = 0;
        $dumpfile("universal shift reg.vcd");
        $dumpvars(0, universal shift reg tb);
        $monitor("t=%0t reset=%b mode=%b serial_in=%b parallel=%b Q=%b serial_out=%b",
                 $time, reset, mode, serial_in, parallel_in, Q, serial_out);

        repeat (2) @(negedge clk);
        reset = 0;

        mode = 2'b11; parallel_in = 4'b1011; @(negedge clk); // load 1011
        mode = 2'b00; @(negedge clk);                        // hold 1011
        mode = 2'b01; serial_in = 1; @(negedge clk);         // right -> 1101
        mode = 2'b10; serial_in = 0; @(negedge clk);         // left -> 1010
        mode = 2'b11; parallel_in = 4'b0100; @(negedge clk);// load 0100
        mode = 2'b00; @(negedge clk);                        // hold 0100
        $finish;
    end
endmodule
```

Simulation Waveform

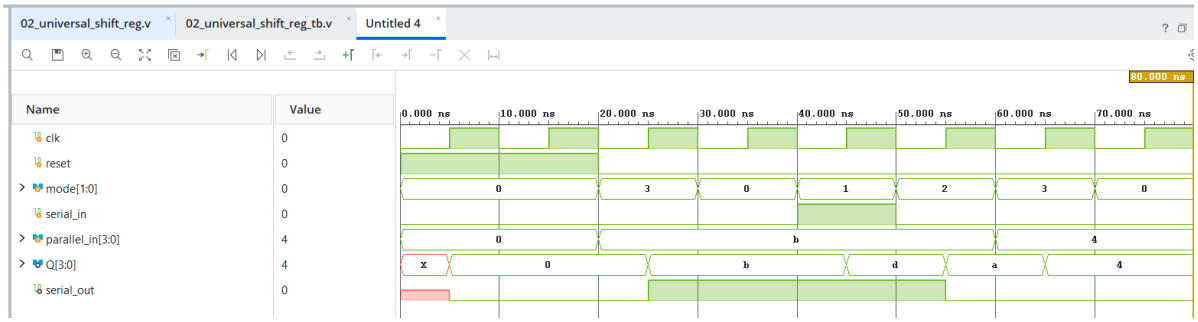


Figure 10.2: Simulation waveform of the 4-Bit Shift Register (Universal)

Observation: The register is first loaded with 1011 (B) and then holds this value. A right shift with serial_in = 1 gives 1101 (D); a left shift with serial_in = 0 gives 1010 (A). Finally 0100 (4) is loaded in parallel and held, as seen on Q in the waveform.

MODULE 11: PARAMETERIZED DIVIDE-BY-N COUNTER

Design Description

The counter counts N rising clock edges and then toggles the output, so the output frequency is the input frequency divided by 2N. N is a parameter (N = 4 in the testbench).

Input / output ports

Port	Direction	Width	Description
N	Parameter	-	Division parameter (N >= 1)
clk	Input	1	Input clock
reset	Input	1	Synchronous active-high reset
clk_out	Output	1	Divided output clock

Circuit Diagram

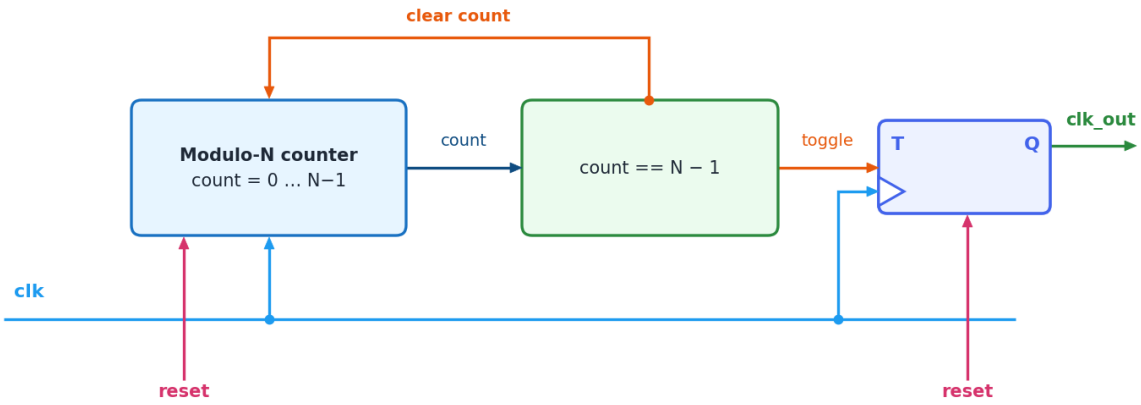


Figure 11.1: Block diagram of the Parameterized Divide-by-N Counter

Verilog HDL Code

```
`timescale 1ns / 1ps

module div by N #(parameter N = 4)(
    input clk,
    input reset,
    output reg clk_out
);
    localparam CW = (N <= 1) ? 1 : $clog2(N);
    reg [CW-1:0] count;

    always @(posedge clk) begin
        if (reset) begin
            count <= 0;
            clk_out <= 0;
        end else if (count == N-1) begin
            count <= 0;
            clk_out <= ~clk_out;
        end else begin
            count <= count + 1'b1;
        end
    end
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module div by N tb;
    reg clk, reset;
    wire clk out;

    div by N #(.N(4)) DUT(.clk(clk), .reset(reset), .clk out(clk out));
    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1;
        $dumpfile("div by N.vcd");
        $dumpvars(0, div by N tb);
        $monitor("t=%0t reset=%b clk=%b clk_out=%b", $time, reset, clk, clk_out);

        #12 reset = 0;
        #240;
        $finish;
    end
endmodule
```

Simulation Waveform



Figure 11.2: Simulation waveform of the Parameterized Divide-by-N Counter

Observation: For N = 4 the output clk_out toggles every four rising edges of clk, so one complete output period equals eight input-clock periods (80 ns for the 10 ns clock), as seen in the waveform.

MODULE 12: MEALY FSM - SEQUENCE DETECTOR (101)

Design Description

In a Mealy machine the output depends on the present state and the input. This FSM detects the overlapping sequence 101 with three states; out = 1 only in S2 when in = 1.

Input / output ports

Port	Direction	Width	Description
clk	Input	1	Clock
reset	Input	1	Synchronous active-high reset
in	Input	1	Serial input bit
out	Output	1	High when 101 has been detected

State transition table

Present state	Next (in = 0)	Next (in = 1)	Output (in = 0 / 1)
S0	S0	S1	0 / 0
S1	S2	S1	0 / 0
S2	S0	S1	0 / 1

Circuit Diagram

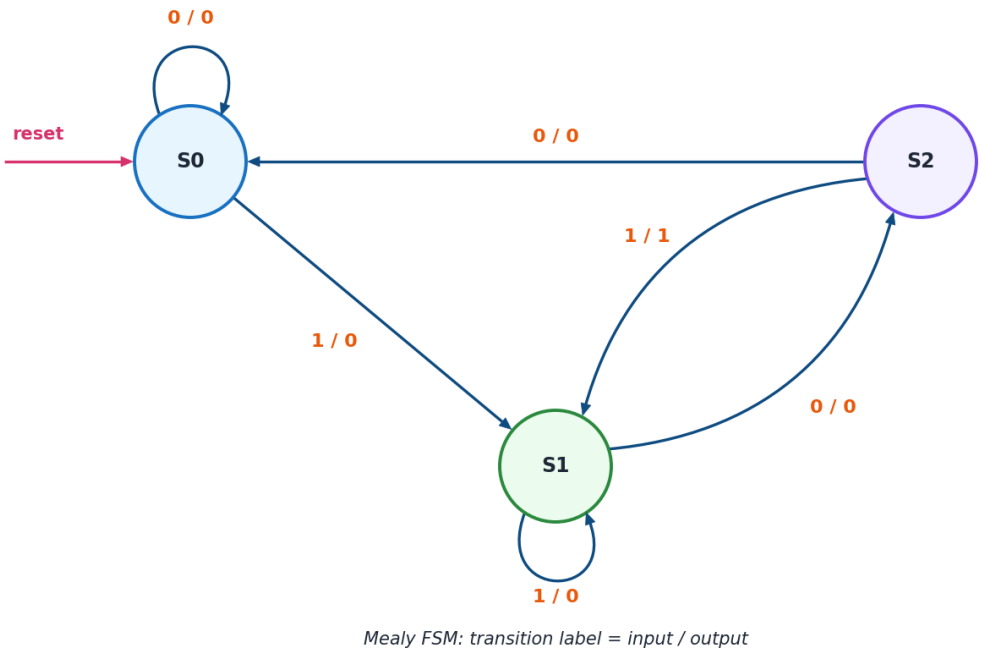


Figure 12.1: State diagram of the Mealy FSM - Sequence Detector (101)

Verilog HDL Code

```
`timescale 1ns / 1ps

module mealy_fsm(
    input clk, reset, in,
    output reg out
);
    reg [1:0] ps_state, nxt_state;
    parameter s0=2'b00, s1=2'b01, s2=2'b10;

    always @(posedge clk) begin
        if (reset) ps_state <= s0;
        else       ps_state <= nxt_state;
    end

    // Next-state logic for sequence 101
    always @(*) begin
        nxt_state = s0;
        case (ps_state)
            s0: if (in) nxt_state = s1; else nxt_state = s0;
            s1: if (in) nxt_state = s1; else nxt_state = s2;
            s2: if (in) nxt_state = s1; else nxt_state = s0;
            default: nxt_state = s0;
        endcase
    end

    // Separate Mealy output logic
    always @(*) begin
        out = 1'b0;
        if ((ps_state == s2) && (in == 1'b1))
            out = 1'b1;
    end
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module mealy_fsm_tb;
    reg clk, reset, in;
    wire out;
    integer errors;

    mealy_fsm DUT(.clk(clk), .reset(reset), .in(in), .out(out));
    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1; in = 0; errors = 0;
        $dumpfile("mealy_fsm.vcd");
        $dumpvars(0, mealy_fsm_tb);
        $monitor("t=%0t reset=%b in=%b state=%b out=%b",
                 $time, reset, in, DUT.ps_state, out);

        repeat (2) @(negedge clk);
        reset = 0;

        // Feed 101; check output shortly after final input is driven
        @(negedge clk); in = 1;
        @(negedge clk); in = 0;
        @(negedge clk); in = 1; #1;
        if (out !== 1'b1) begin
            $display("FAIL: 101 was not detected"); errors = errors + 1;
        end else $display("PASS: 101 detected");

        // Feed 00 after detection; no new detection
        @(negedge clk); in = 0;
        @(negedge clk); in = 0; #1;
        if (out !== 1'b0) begin
            $display("FAIL: unexpected output"); errors = errors + 1;
        end else $display("PASS: no false detection");

        if (errors == 0) $display("ALL TESTS PASSED");
        else $display("TESTS FAILED: %0d", errors);
        $finish;
```

```
    end
endmodule
```

Simulation Waveform

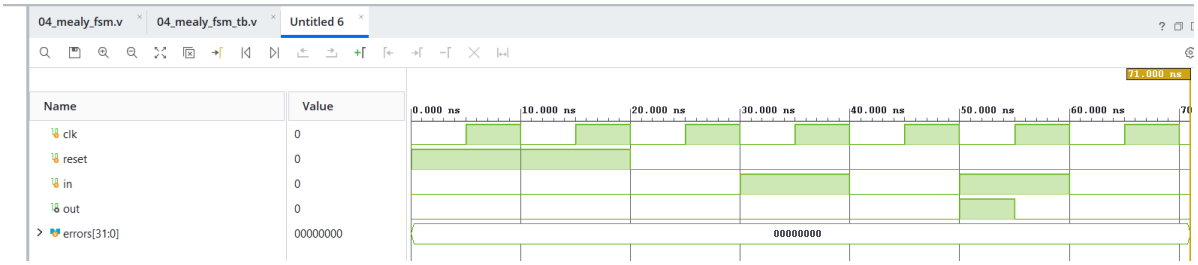


Figure 12.2: Simulation waveform of the Mealy FSM - Sequence Detector (101)

Observation: When the bits 1, 0, 1 are applied, out rises in the same clock period in which the final 1 is present at the input (Mealy behaviour: the output responds to the input without waiting for a clock edge). It is low for all other inputs, and the testbench reports errors = 0.

MODULE 13: MOORE FSM - SEQUENCE DETECTOR (101)

Design Description

In a Moore machine the output depends only on the present state. This FSM detects the overlapping sequence 101 with four states; out = 1 only in state S3.

Input / output ports

Port	Direction	Width	Description
clk	Input	1	Clock
reset	Input	1	Synchronous active-high reset
in	Input	1	Serial input bit
out	Output	1	High in the detection state S3

State transition table

Present state	Next (in = 0)	Next (in = 1)	Output
S0	S0	S1	0
S1	S2	S1	0
S2	S0	S3	0
S3	S2	S1	1

Circuit Diagram

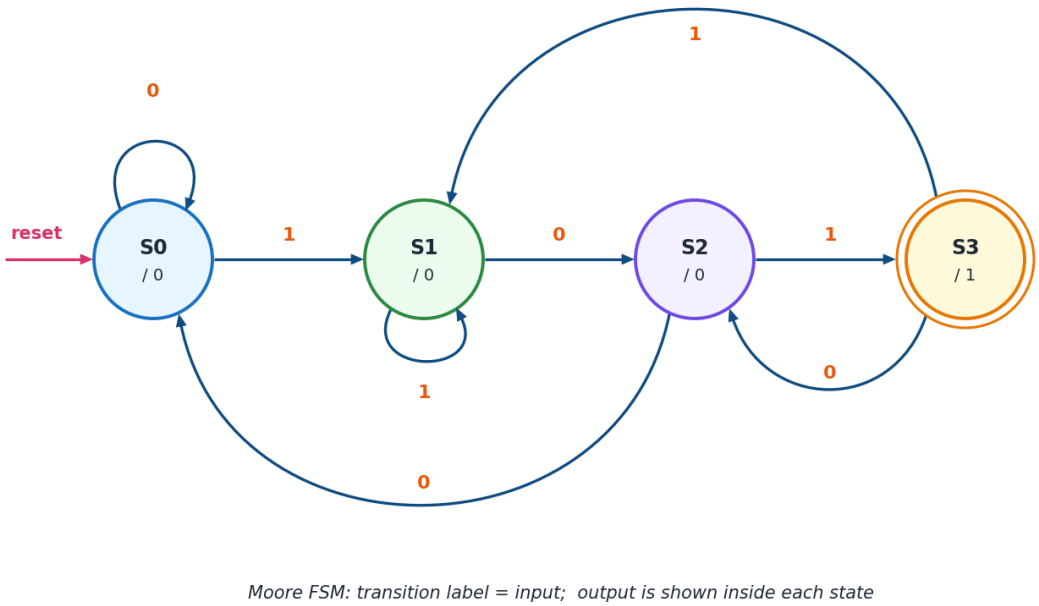


Figure 13.1: State diagram of the Moore FSM - Sequence Detector (101)

Verilog HDL Code

```
`timescale 1ns / 1ps

module moore_fsm(
    input clk, reset, in,
    output reg out
);
    reg [1:0] ps_state, nxt_state;
    parameter s0=2'b00, s1=2'b01, s2=2'b10, s3=2'b11;

    always @(posedge clk) begin
        if (reset) ps_state <= s0;
        else       ps_state <= nxt_state;
    end

    // Next-state logic for sequence 101
    always @(*) begin
        nxt_state = s0;
        case (ps_state)
            s0: if (in) nxt_state = s1; else nxt_state = s0;
            s1: if (in) nxt_state = s1; else nxt_state = s2;
            s2: if (in) nxt_state = s3; else nxt_state = s0;
            s3: if (in) nxt_state = s1; else nxt_state = s2;
            default: nxt_state = s0;
        endcase
    end

    // Moore output depends only on current state
    always @(*) begin
        if (ps_state == s3) out = 1'b1;
        else                out = 1'b0;
    end
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module moore_fsm_tb;
    reg clk, reset, in;
    wire out;
    integer errors;

    moore_fsm DUT(.clk(clk), .reset(reset), .in(in), .out(out));
    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1; in = 0; errors = 0;
        $dumpfile("moore_fsm.vcd");
        $dumpvars(0, moore_fsm_tb);
        $monitor("t=%0t reset=%b in=%b state=%b out=%b",
                 $time, reset, in, DUT.ps_state, out);

        repeat (2) @(negedge clk);
        reset = 0;

        // Sequence 101
        @(negedge clk); in = 1;
        @(negedge clk); in = 0;
        @(negedge clk); in = 1;
        @(negedge clk); #1;
        if (out !== 1'b1) begin
            $display("FAIL: 101 not detected"); errors = errors + 1;
        end else $display("PASS: 101 detected");

        // Leave detection state
        @(negedge clk); in = 0; #1;
        if (out !== 1'b0) begin
            $display("FAIL: output did not clear"); errors = errors + 1;
        end else $display("PASS: output cleared");

        if (errors == 0) $display("ALL TESTS PASSED");
        else $display("TESTS FAILED: %0d", errors);
        $finish;
```

```
    end
endmodule
```

Simulation Waveform

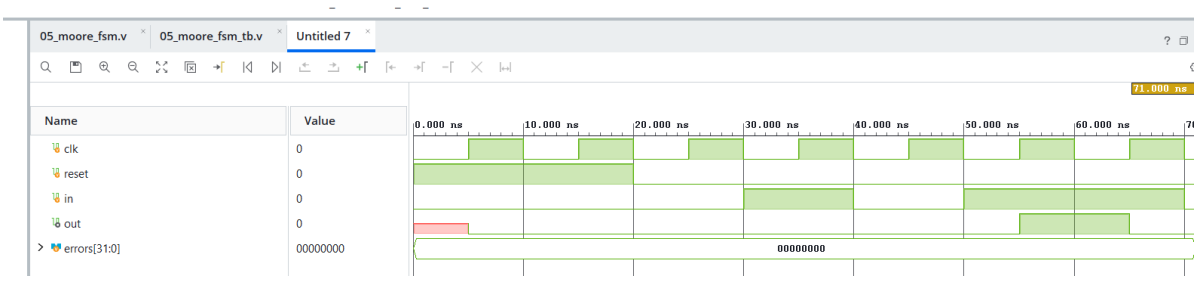


Figure 13.2: Simulation waveform of the Moore FSM - Sequence Detector (101)

Observation: After the bits 1, 0, 1 are received, the FSM moves to S3 on the next clock edge and out goes high for one clock period; it returns low when the next input arrives. The output changes only at clock edges, which distinguishes it from the Mealy detector. The testbench reports errors = 0.

MODULE 14: RANDOM SEQUENCE GENERATOR (4-BIT LFSR)

Design Description

The 4-bit LFSR uses the polynomial x^4 + x^3 + 1 (feedback = Q3 XOR Q2) and cycles through all 15 non-zero states. Reset loads the seed 0001.

Input / output ports

Port	Direction	Width	Description
clk	Input	1	Clock
reset	Input	1	Synchronous reset (loads seed 0001)
q[3:0]	Output	4	Parallel pseudo-random output
rand_bit	Output	1	Serial pseudo-random bit (q[3])

Circuit Diagram

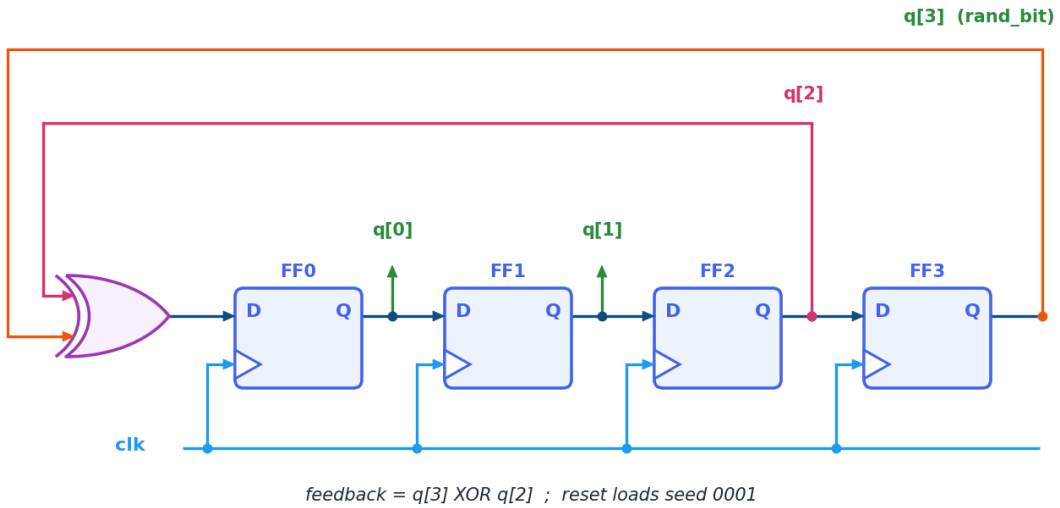


Figure 14.1: Circuit diagram of the Random Sequence Generator (4-Bit LFSR)

Verilog HDL Code

```
`timescale 1ns / 1ps

// 4-bit maximal-length LFSR, polynomial x^4 + x^3 + 1.
// Cycles through 15 non-zero states (the all-zero state is never reached).
module lfsr_4bit(
    input clk, reset,
    output reg [3:0] q,
    output rand bit
);
    wire feedback;
    assign feedback = q[3] ^ q[2];
    assign rand_bit = q[3];

    always @(posedge clk) begin
        if (reset) q <= 4'b0001;             // non-zero seed
        else       q <= {q[2:0], feedback};
    end
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module lfsr_4bit_tb;
    reg clk, reset;
    wire [3:0] q;
    wire rand_bit;
    reg [3:0] seq [0:14];
    integer i, errors;

    lfsr_4bit DUT(.clk(clk), .reset(reset), .q(q), .rand_bit(rand_bit));
    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1; errors = 0;
        seq[0]  = 4'b0001; seq[1]  = 4'b0010; seq[2]  = 4'b0100; seq[3]  = 4'b1001;
        seq[4]  = 4'b0011; seq[5]  = 4'b0110; seq[6]  = 4'b1101; seq[7]  = 4'b1010;
        seq[8]  = 4'b0101; seq[9]  = 4'b1011; seq[10] = 4'b0111; seq[11] = 4'b1111;
        seq[12] = 4'b1110; seq[13] = 4'b1100; seq[14] = 4'b1000;
        $dumpfile("lfsr_4bit.vcd");
        $dumpvars(0, lfsr_4bit_tb);
        $monitor("t=%0t reset=%b q=%b rand_bit=%b", $time, reset, q, rand_bit);

        repeat (2) @(negedge clk);
        reset = 0;

        // Check two full periods (the sequence repeats after 15 clocks)
        for (i = 0; i < 31; i = i + 1) begin
            #1;
            if (q !== seq[i % 15]) begin
                $display("FAIL: step %0d expected %b got %b", i, seq[i % 15], q);
                errors = errors + 1;
            end
            @(negedge clk);
        end

        if (errors == 0) $display("ALL TESTS PASSED");
        else $display("TESTS FAILED: %0d", errors);
        $finish;
    end
endmodule
```

Simulation Waveform



Figure 14.2: Simulation waveform of the Random Sequence Generator (4-Bit LFSR)

Observation: The output follows the sequence 1, 2, 4, 9, 3, 6, D, A, 5, B, 7, F, E, C, 8 (hexadecimal) and then repeats from 1, covering all 15 non-zero states exactly once per period. The testbench checks two complete periods and reports errors = 0.